



EPS-16 PLUS
Digital Sampling Workstation
External Command Specification

Version 1.30.00

Table of Contents:

<u>Section number:</u>	<u>Page:</u>
1 <u>Introduction and Overview</u>	4
2 <u>MIDI System Exclusive (SYSEX) Packet Pieces</u>	5
2.1 MIDI System Exclusive Packet Header	5
2.2 MIDI System Exclusive Packet Tail	5
2.3 Message Formats	5
12 Bit Word Format	5
16 Bit Word Format	5
3 <u>Rules of the Game</u>	6
3.1 General Command Exchange (NORMAL mode)	6
3.2 Special WAIT Command Exchange (WAIT mode)	6
4 <u>Command List</u>	7
Command Description format	7
4.1 General Communication Commands	7
00 (unused command code)	7
01 RESPONSE	7
02 CANCEL	7
4.2 Parameter and WaveSample Data Transfer Commands	8
NOTE on WaveData Offsets	8
03 GET INSTRUMENT	8
04 GET LAYER	8
05 GET WAVESAMPLE PARAMETERS	8
06 GET WAVESAMPLE DATA	9
07 GET PITCH TABLE	9
08 GET PARAMETER	9
09 (unused command code)	9
0A GET WAVESAMPLE OVERVIEW	10
0B (unused command code)	10
Note on PUT Commands	11
0C PUT INSTRUMENT	11
0D PUT LAYER	11
0E PUT WAVESAMPLE PARAMETERS	11
0F PUT WAVESAMPLE DATA	12
10 PUT PITCH TABLE	12
11 PUT PARAMETER	13
13 PUT WAVESAMPLE OVERVIEW	13
4.3 Instrument Editing Commands	14
14 SAVE INSTRUMENT	14
15 CREATE INSTRUMENT	14
1C DELETE INSTRUMENT	14
12 COPY INSTRUMENT	15
16 CREATE LAYER	16
17 DELETE LAYER	16
18 COPY LAYER	16
19 CREATE WAVESAMPLE	17
1A DELETE WAVESAMPLE	17
1B COPY WAVESAMPLE	17
1D AUDITION WAVESAMPLES	18
40 VIRTUAL BUTTON PRESS	18
44 CREATE PRESET	18

Commands 30 - 3F not specified

<u>Section number:</u>	<u>Page:</u>
4.4 WaveSample Digital Sample Processing Commands	19
NOTE on Scale Factors	19
NOTE on Fade Zones	19
1E TRUNCATE WAVESAMPLE	19
1F CLEAR WAVEDATA	19
20 COPY WAVEDATA	20
21 ADD WAVEDATA	21
22 SCALE WAVEDATA	22
23 INVERT WAVEDATA	22
24 REVERSE WAVEDATA	23
25 REPLICATE WAVEDATA	23
26 CROSS FADE LOOP	24
27 FADE IN WAVEDATA	24
28 FADE OUT WAVEDATA	25
29 REVERSE CROSS FADE LOOP	25
2A ENSEMBLE CROSS FADE LOOP	26
2B BOWTIE CROSS FADE LOOP	26
2C LENGTHEN LOOP	27
2D MIX WAVEDATA	27
2E MERGE/SPLICE WAVEDATA	28
2F VOLUME SMOOTHING	29
41 NORMALIZE GAIN	29
42 SYNTHESIZED LOOP	30
43 BIDIRECTIONAL CROSS FADE LOOP	30
5 Response Command Status Codes	31
6 Button Numbers (for Virtual Button Press Commands)	33
7 Parameter Block Data Descriptions	34
7.1 Instrument Parameter Block	34
7.2 Layer Parameter Block	35
7.3 WaveSample Parameter Block	36
7.3.1 WaveSample Envelope Description	38
7.4 Pitch Table Block	38
8 Examples of Data Transfer Protocol	39
9 Parameter Numbers	40
9.1 SYSTEM•MIDI PARAMETERS	40
9.2 SEQUENCE•SONG PARAMETERS	41
9.2.1 TRACK PARAMETERS	41
9.3 INSTRUMENT PARAMETERS	42
9.4 LAYER PARAMETERS	42
9.5 WAVESAMPLE PARAMETERS	43
9.6 ENVELOPE PARAMETERS	43
9.7 PITCH PARAMETERS	44
9.8 FILTER PARAMETERS	44
9.9 AMP (VOLUME) PARAMETERS	44
9.10 LFO PARAMETERS	45
9.11 EDIT PARAMETERS	45
9.12 EFFECTS PARAMETERS	46
10 PARAMETER VALUE TABLES	51
Appendix A: EPS-16 PLUS SCSI Data Transfer Protocol	55
Appendix B: A Quick Description of Instrument Format	56

ENSONIQ EPS-16 PLUS Digital Sampling Workstation External Command Specification

Version 1.30.00

1 Introduction and Overview

This manual describes the communication protocol and external commands used when the EPS-16 PLUS is communicating with an external computer (EXT). The protocol is designed to aid the implementation of remote editing software running on EXT, and as such is especially relevant to computer software designers and programmers.

Please NOTE: This document supersedes all previous EPS External Command Specification documentation, and applies specifically to the EPS-16 PLUS only (not the Original EPS).

The commands described herein allow external visual editing programs to collect information about instruments, layers, and WaveSamples within the EPS-16 PLUS, to alter parameters, and to process WaveSample data. Since the primary function of a visual editor is viewing data, most existing editors spend a lot of time transferring data between EXT and the EPS-16 PLUS. Also, in many existing editors, one machine will process the data while the other waits on the processing machine before transferring the data to see the results. The data processing commands described in this manual save editing time by allowing the EPS-16 PLUS and EXT to independently process data and achieve the same results without the need for lots of data transfers. The GET WAVESAMPLE OVERVIEW command is designed to allow a visual editor running on an EXT with few memory resources to view EPS-16 PLUS WaveSample data without the need for EXT to store the data internally.

The protocol can be used in either MIDI System Exclusive or SCSI (Small Computer Systems Interface) messages. When an SCSI cable is connected between EXT and the EPS-16 PLUS, all data transfers to and from sound RAM (parameter and WaveData blocks) will occur over SCSI. See **Appendix A** for details on implementing the protocol using an SCSI interface.

2 MIDI System Exclusive (SYSEX) Packet Pieces

A packet is a bunch of information, i.e. a message, in the form of a data stream. Each packet can be divided into three sections or pieces. The first and last packet pieces form the frame for a message. The <Message> contains the commands described in Section 4. Every <Message> must be preceded by a SYSEX header and followed by a SYSEX tail. A complete packet looks like this:

SYSEX Header...<Message>...SYSEX Tail

2.1 MIDI System Exclusive Packet Header

This is the common system exclusive header which must be used on all SYSEX messages to and from the EPS-16 PLUS. These four bytes are always sent preceding the message.

<u>binary</u>	<u>hex</u>	
11110000	F0	System Exclusive status byte
00001111	0F	ENSONIQ manufacturer Code
00000011	03	EPS product ID Code
0000nnnn	0x	nnnn = x = Base MIDI channel number (0-15) (Edit/System•Midi, BASE CHANNEL minus 1)

2.2 MIDI System Exclusive Packet Tail

For every head there is a tail. This byte follows every message.

11110111	F7	End of System Exclusive
----------	----	-------------------------

NOTE: When displayed by a computer, hexadecimal (hex) numbers are often preceded by a "\$" sign in order to differentiate them from regular decimal numbers.

2.3 Message Formats

The EPS-16 PLUS message format within the packet frame consists of a command byte followed by any number of data words. There are two word formats implemented in order to speed up message transfer by minimizing unnecessary data bytes.

12 Bit Word Format

The 12 bit format is used for data in most of the messages and consists of two 6 bit MIDI bytes. The 12 bits are right justified within a 16 bit register. The 12 least significant bits of the 16 bit word 0000xxxxxyyyyyy would be transmitted as follows:

00xxxxxx	xxxxxx = Data word hi byte
00yyyyyy	yyyyyy = Data word lo byte

16 Bit Word Format

The 16 bit word format, which consists of three MIDI bytes, is used when all 16 bits of a word need to be transmitted. The 16 bit word HHHHhhhhh111111 would be transmitted as follows:

0000HHHH	HHHH = Hi 4 bits of data word
00hhhhh	hhhhh = Middle 6 bits of data word
00111111	11111 = Lo 6 bits of data word

3 Rules of the Game

These rules describe the communications protocol between the transmitter and the receiver. In general, for each command message sent by the transmitter, there is a response command sent by the receiver. Either device (EPS-16 PLUS or EXT) can be transmitter and receiver simultaneously, so that each one can be processing a command that it received while waiting for a response to a command that it sent. However, a transmitter cannot send another command before receiving a response to a previous command, except in WAIT mode, as described in Section 3.2.

3.1 General Command Exchange (NORMAL mode)

This set of rules describes how 99% of the commands should be processed. These rules allow an open or closed loop between the transmitter and receiver. An open loop exists when there is one MIDI cable from the transmitter to the receiver, so that the transmitter will not receive response commands from the receiver. A closed loop uses two MIDI cables between the transmitter and receiver, so that two-way communication is possible. A closed loop is more efficient since the transmitter can resume its operation immediately after receiving a response command, rather than always waiting for a timeout.

- The transmitter sends the message containing a command and parameters (if applicable) and starts the 2 second command timer. The transmitter should not send another command until either it gets a response from the receiver or it times out after 2 seconds. A few specific commands that do not need responses are noted in Section 4, but in these cases the transmitter must still wait 2 seconds before sending another command.
- The receiver processes the command and sends a response command with the proper status code. The receiver should either ignore all other commands while it is processing a command, or it should send a response command with a negative acknowledge (NAK) status code if another message is received during processing. See Section 4.1 for the response command format, and Section 5 for the list of response command status codes.
- If the transmitter times out after 2 seconds, either the link may be broken or there is an open loop. The transmitter must use its own discretion when processing a timeout error. If the EPS-16 PLUS times out while waiting for an ACK from EXT, it will not proceed with whatever dump was requested. See Section 8 for an example of data transfer protocol between EXT and the EPS-16 PLUS.

3.2 Special WAIT Command Exchange (WAIT mode)

This set of rules should be followed when the receiver sends a response command with the WAIT status code (see Section 5 for status codes), thus initiating WAIT mode.

- The transmitter either restarts the command timer with a 30 second value and sends a response command with the ACK status code, or it sends the CANCEL command (see Section 4.1). The receiver should wait for the transmitter's response before doing whatever it is that requires the WAIT mode.
- If the transmitter sent the CANCEL command, the receiver should abort the original command and send a response command with the ACK status code.
- If the transmitter sent an ACK, the receiver should process the original command and send the appropriate response command as soon as possible.
- If the receiver needs more time, it can send a response command with the WAIT status code again. The transmitter should then re-enter wait mode by resetting the 30 second command timer.
- If the transmitter times out, it must use its own discretion when handling the error. An error message to the user or a longer wait are two possible timeout error responses.

4 Command List

This is the list of commands to be used in all messages communicated between EXT and the EPS-16 PLUS. These command descriptions contain the detailed content of the <Message> enclosed within the system exclusive (SYSEX) packet frame described in Section 2. Unless otherwise specified, all command parameter words use the 12 bit word format as described in Section 2.3. All unused command codes should be considered reserved for future use. ^ p.5

Command Description format

The command code is shown as a **bold** hexadecimal number, followed by the CAPITALIZED name of the command. The individual parameters of each command are described in the order that they are found in the message.

4.1 General Communication Commands

00 (unused command code)

01 RESPONSE

Send response to a received command.

00	Status Code hi byte, (always 0)
0a	a = Status Code lo byte, (see Section 5 for a description of the Status Codes)

^ p. 31

02 CANCEL

Abort current command processing. This is only valid in WAIT mode (it should be ignored in NORMAL mode; see Section 3.2)

^ p. 6

4.2 Parameter and WaveSample Data Transfer Commands

This set of commands is used to GET and PUT parameter values, and to transfer WaveSample data back and forth between EXT and the EPS-16 PLUS. A GET command transmitted by EXT will prompt the EPS-16 PLUS to transmit a PUT command. If EXT then sends an ACK before the 2 second timeout limit, the EPS-16 PLUS will respond by sending the data block. See **Section 8** for an example of data transfer protocol between EXT and the EPS-16 PLUS. *^ p.39*

NOTE: The EPS-16 PLUS does not transmit GET commands (commands 03 to 0A).

NOTE on WaveData Offsets: WaveData offset address parameters are used internally as long words (20 bits). They are transmitted as two 12 bit words. For example, the offset 00200 hex would be sent as the four parameter bytes 00, 00, 08, and 00 hex. Likewise, the offset 0AAAA hex would be sent as 00, 0A, 2A, and 2A hex.

NOTE: All parameter and WaveSample data transfer commands contain the current edit context of edit instrument, layer, and WaveSample.

03 GET INSTRUMENT

Request for instrument parameters.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, Layer # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]

04 GET LAYER

Request for layer parameters.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, Layer # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]

*} eg WaveSample 10 = 0A
00
" 20 = 0F
04*

05 GET WAVESAMPLE PARAMETERS

Request for WaveSample parameters.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, Layer # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]

06 GET WAVESAMPLE DATA

Request for WaveSample data. See **Section 8** for an example of data transfer protocol between EXT and the EPS-16 PLUS. *^ p. 39*

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, Layer # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]
0e	Data range start offset (from beginning of
0f	WaveSample data), Data offset = [0..FFFFFF]
0g	Start Offset = 0efgh (long word, 20 bits)
0h	(see note at top of Section 4.2) — <i>p. 8</i>
0i	Data range end offset (from beginning of
0j	WaveSample data), Data offset = [0..FFFFFF]
0k	End Offset = 0ijkl (long word, 20 bits)
0l	(see note at top of Section 4.2) — <i>p. 8</i>

Start offset

End offset

07 GET PITCH TABLE

Request for the pitch table data within the requested layer.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, Layer # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]

08 GET PARAMETER

Request for a parameter value. See **Section 9** for the parameter numbers. *p. 40*

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, Layer # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]
0e	e = Parameter number hi byte
0f	f = Parameter number lo byte

09 (unused command code)

0A GET WAVESAMPLE OVERVIEW

Request for WaveSample data overview. This is never transmitted by the EPS-16 PLUS.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, Layer # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]
0e	Data range start offset (from beginning of
0f	WaveSample data), Data offset = [0..FFFFFF]
0g	Start Offset = 0efgh (long word, 20 bits)
0h	(see note at top of Section 4.2) p. 8
0i	Data range end offset (from beginning of
0j	WaveSample data), Data offset = [0..FFFFFF]
0k	End Offset = 0ijkl (long word, 20 bits)
0l	(see note at top of Section 4.2)

Λ p. 8

0B (unused command code)

Note on PUT Commands: All PUT commands that contain parameter blocks or WaveData are transmitted in two messages. The first message contains the command code and the edit context (instrument, layer, and WaveSample). The second message is only the data block. If an ACK is received after the first piece has been sent, but before timeout, the second piece will be transmitted as described in **Section 3.1**. If an ACK is not received before timeout, the EPS-16 PLUS will not transmit the second piece of the message. p.6

Command 11, PUT PARAMETER, is an exception to this rule, as noted below. See p.34 **Section 8** for an example of data transfer protocol between EXT and the EPS-16 PLUS. See **Section 7** for descriptions of the parameter block data in the second message. ^ p.34

NOTE: All parameter and WaveSample data transfer commands contain the current edit context of edit instrument, layer, and WaveSample.

0C PUT INSTRUMENT

Instrument parameter block dump.

```
00      Instrument number hi byte (always 0)
0a      a = Instrument number lo byte, # = [0..7]
00      Layer number hi byte (always 0)
0b      b = Layer number lo byte, Layer # = [0..7]
0c      c = WaveSample number hi byte
0d      d = WaveSample number lo byte, # = [1..127]
```

Instrument parameter data block as described in **Section 7.1**. p.34
(using the 16 bit word format described in **Section 2.3**)
^ p.5

0D PUT LAYER

Layer parameter block dump.

```
00      Instrument number hi byte (always 0)
0a      a = Instrument number lo byte, # = [0..7]
00      Layer number hi byte (always 0)
0b      b = Layer number lo byte, Layer # = [0..7]
0c      c = WaveSample number hi byte
0d      d = WaveSample number lo byte, # = [1..127]
```

Layer parameter data block as described in **Section 7.2**. p.35
(using the 16 bit word format described in **Section 2.3**)
^ p.5

0E PUT WAVESAMPLE PARAMETERS

WaveSample parameter block dump.

```
00      Instrument number hi byte (always 0)
0a      a = Instrument number lo byte, # = [0..7]
00      Layer number hi byte (always 0)
0b      b = Layer number lo byte, Layer # = [0..7]
0c      c = WaveSample number hi byte
0d      d = WaveSample number lo byte, # = [1..127]
```

WaveSample parameter data block as described in **Section 7.3**. p.36
(using the 16 bit word format described in **Section 2.3**)
^ p.5

0F PUT WAVESAMPLE DATA

WaveSample data block dump. See the example of data transfer protocol in **Section 8.** p. 39

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, Layer # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]
0e	Data range start offset (from beginning of
0f	WaveSample data), Data offset = [0..FFFFFF]
0g	Start Offset = 0efgh (long word, 20 bits)
0h	(see note at top of Section 4.2) p. 8
0i	Data range end offset (from beginning of
0j	WaveSample data), Data offset = [0..FFFFFF]
0k	End Offset = 0ijkl (long word, 20 bits)
0l	(see note at top of Section 4.2) p. 8

All 16 bit signed sample data words within the data range.
(using the 16 bit word format described in **Section 2.3**)

^ p. 5

10 PUT PITCH TABLE

Transmit the pitch table data for the specified layer.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, Layer # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]

Pitch table block described in **Section 7.4.** p. 38
(using the 16 bit word format described in **Section 2.3**)

^ p. 5

11 PUT PARAMETER

Update a parameter value for an instrument, layer, or WaveSample. If a System•MIDI parameter is being PUT, the instrument, layer, and WaveSample values are irrelevant, but still must be included in the message; it is recommended that they be transmitted as all zeros. See **Section 9** for the parameter numbers. See **Section 8** for an example of a PUT PARAMETER message.

NOTE: PUT PARAMETER commands do not require a response. A response command status code is only sent if the parameter number or value is erroneous.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, Layer # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]
52 — 0e	e = Parameter number hi byte
5 0f	f = Parameter number lo byte, # = [0..255]
0g	g = Parameter value hi byte of hi word
0h	h = Parameter value lo byte of hi word
0i	i = Parameter value hi byte of lo word
0j	j = Parameter value lo byte of lo word

13 PUT WAVESAMPLE OVERVIEW

WaveSample data overview dump. **NOTE:** This is not received by the EPS-16 PLUS.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, Layer # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]
0e	Data range start offset (from beginning of
0f	WaveSample data), Data offset = [0..FFFFFF]
0g	Start Offset = 0efgh (long word, 20 bits)
0h	(see NOTE at top of Section 4.2)
0i	Data range end offset (from beginning of
0j	WaveSample data), Data offset = [0..FFFFFF]
0k	End Offset = 0ijkl (long word, 20 bits)
0l	(see NOTE at top of Section 4.2)

512 data words.

(The data range is divided into 512 sections, and the maximum 16 bit absolute sample value within each section is transmitted using the 16 bit word format described in **Section 2.3**.)

4.3 Instrument Editing Commands

This set of commands is used when editing an instrument, and any of the objects within an instrument, including layers and WaveSamples.

NOTE: All instrument editing commands contain the current edit context of edit instrument, layer, and WaveSample.

14 SAVE INSTRUMENT

Save an Instrument to the current directory of the current storage device. Using this command will always overwrite any file in the current directory that has the same name as the Instrument being saved, without any warning or prompt. To rename the Instrument before saving, first retrieve the Instrument parameter data block from the EPS-16 PLUS with command 03, GET INSTRUMENT. Next, edit the Instrument Name data (see **Section 7.1**) and then transmit the edited Instrument parameter data block back to the EPS-16 PLUS with command 0C, PUT INSTRUMENT. See **Section 4.2** for the GET and PUT command message format.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, Layer # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]

15 CREATE INSTRUMENT

Create a new instrument with no layers or WaveSamples. The instrument must not already exist.

00	New instrument number hi byte (always 0)
0a	a = New instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, Layer # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]

1C DELETE INSTRUMENT

Delete the instrument and free up memory in the EPS-16 PLUS. The instrument must already exist.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, Layer # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]

12 COPY INSTRUMENT

Copy the source instrument to the destination instrument. The source instrument must already exist, and the destination must not already exist.

00	Source instrument number hi byte (always 0)
0a	a = Source instrument number lo byte, # = [0..7]
00	Source layer number hi byte (always 0)
0b	b = Source layer number lo byte, # = [0..7]
0c	c = Source WaveSample number hi byte
0d	d = Source WaveSample number lo byte, # = [1..127]
00	Destination instrument number hi byte (=0)
0e	e = Destination instrument number lo byte
00	Destination layer number hi byte (always 0)
0f	f = Destination layer number lo byte, # = [0..7]
00	Dummy destination WaveSample hi byte.
00	Dummy destination WaveSample lo byte.

16 CREATE LAYER

Define a new layer with one WaveSample in an instrument. The new layer must not already exist.

```
00      Instrument number hi byte (always 0)
0a      a = Instrument number lo byte, # = [0..7]
00      New layer number hi byte (always 0)
0b      b = New layer number lo byte, # = [0..7]
0c      c = WaveSample number hi byte
0d      d = WaveSample number lo byte, # = [1..127]
```

17 DELETE LAYER

Undefine a layer in an instrument. All WaveSamples in the layer are deleted too. The layer must already exist.

```
00      Instrument number hi byte (always 0)
0a      a = Instrument number lo byte, # = [0..7]
00      Layer number hi byte (always 0)
0b      b = Layer number lo byte, Layer # = [0..7]
0c      c = WaveSample number hi byte
0d      d = WaveSample number lo byte, # = [1..127]
```

18 COPY LAYER

Copy the source layer to the destination layer.

```
00      Source instrument number hi byte (always 0)
0a      a = Source instrument number lo byte, # = [0..7]
00      Source Layer number hi byte (always 0)
0b      b = Source Layer number lo byte, # = [0..7]
0c      c = Source WaveSample number hi byte
0d      d = Source WaveSample number lo byte, # = [1..127]
00      Destination instrument number hi byte (=0)
0e      e = Destination instrument number lo byte
00      Destination layer number hi byte (always 0)
0f      f = Destination layer number lo byte, # = [0..7]
00      Dummy destination WaveSample hi byte.
00      Dummy destination WaveSample lo byte.
00      Copy data flag hi byte. (always 0)
0g      g = Copy data flag lo byte. When flag is set, data will be copied (clear =
00, set = 01)
```

19 CREATE WAVESAMPLE

Create a new WaveSample in a layer with a single cycle square wave for data. The WaveSample must not already exist.

```
00      Instrument number hi byte (always 0)
0a      a = Instrument number lo byte, # = [0..7]
00      Layer number hi byte (always 0)
0b      b = Layer number lo byte, Layer # = [0..7]
0c      c = New WaveSample number hi byte
0d      d = New WaveSample number lo byte, # = [1..127]
```

1A DELETE WAVESAMPLE

Undefine a WaveSample in a layer and free up RAM. The WaveSample must already exist.

```
00      Instrument number hi byte (always 0)
0a      a = Instrument number lo byte, # = [0..7]
00      Layer number hi byte (always 0)
0b      b = Layer number lo byte, Layer # = [0..7]
0c      c = WaveSample number hi byte
0d      d = WaveSample number lo byte, # = [1..127]
```

1B COPY WAVESAMPLE

Copy WaveSample parameters from the source WaveSample to a new WaveSample on the destination layer. The WaveData is copied if the data copy flag is set.

```
00      Source instrument number hi byte (always 0)
0a      a = Source instrument number lo byte, # = [0..7]
00      Source layer number hi byte (always 0)
0b      b = Source layer number lo byte, # = [0..7]
0c      c = Source WaveSample number hi byte
0d      d = Source WaveSample number lo byte, # = [1..127]
00      Destination instrument number hi byte (=0)
0e      e = Destination instrument number lo byte
00      Destination layer number hi byte (always 0)
0f      f = Destination layer number lo byte, # = [0..7]
0g      g = Destination WaveSample number hi byte
0h      h = Destination WaveSample number lo byte
00      Copy data flag hi byte, (always 0)
0i      i = Copy data flag lo byte. When flag is set, data will be copied (clear = 00,
      set = 01)
```

1D AUDITION WAVESAMPLES

Audition an OLD and NEW WaveSample using the audition utilities of the EPS-16 PLUS. After the command is executed, EPS-16 PLUS will send a Delete WaveSample command specifying the WaveSample that was not kept. The EPS-16 PLUS never transmits this command.

00	Old instrument number hi byte (always 0)
0a	a = Old instrument number lo byte, # = [0..7]
00	Old layer number hi byte (always 0)
0b	b = Old layer number lo byte, # = [0..7]
0c	c = Old WaveSample number hi byte
0d	d = Old WaveSample number lo byte, # = [1..127]
00	New instrument number hi byte (always 0)
0e	e = New instrument number lo byte, # = [0..7]
00	New layer number hi byte (always 0)
0f	f = New layer number lo byte, # = [0..7]
0g	g = New WaveSample number hi byte
0h	h = New WaveSample number lo byte, # = [1..127]

40 VIRTUAL BUTTON PRESS

Send a Virtual Button Press. This should only be used if the desired operation cannot be performed using the existing commands. See **Section 6** for the list of valid button numbers, and an example of a Virtual Button Press message.

00	Button number hi byte. (always 0)
0a	a = Button number lo byte. # = [see table in Section 6] p. 33

44 CREATE PRESET

Create a preset with the current selection of instruments and patches. Refer to the EPS-16 PLUS Musician's Manual for more information on presets.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, Layer # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]
00	Preset number hi byte (always 0)
0e	e = Preset number lo byte, # = [0..7]

4.4 WaveSample Digital Sample Processing Commands

These commands alter WaveSample data using the EPS-16 PLUS Digital Sample Processing functions. See the EPS-16 PLUS Musician's Manual for details on how these functions alter the WaveSample data.

NOTE on WaveData Offsets: See the note at the top of Section 4.2 for more information on WaveData offsets.

NOTE on Scale Factors: Scale factors are stored internally as a 16 bit word, where the hi byte is an integer [0..255] and the lo byte is a 7 bit fraction [0..127]. The factor is transmitted using two words in the 12 bit format so that all 16 bits of the value can be sent. For example, a scale factor of 255.99 (hexadecimal FF7F) is transmitted by the four bytes 00, 0F, 3D, and 3F hex. Likewise, a scale factor of 1.0 (hexadecimal 0100) is transmitted by 00, 00, 04, and 00. The 16 bit scale factor must be converted to two 12 bit numbers before being transmitted, making them similar to transmitting 20 bit data offsets using the 12 bit format.

NOTE on Fade Zones: The cross fade zone value must not be greater than the loop length, nor greater than the length of data preceding the loop or following the loop. Thus, $FZ \leq (LE-LS)$, $FZ \leq (LS-WS)$, and $FZ \leq (WE-LE)$ where FZ is the cross fade zone, WS is the start of the WaveData, WE is the end of the WaveData, LS is the start of the loop, and LE is the end of the loop.

1E TRUNCATE WAVESAMPLE

Delete any unused WaveSample data before sample start and after sample end.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, Layer # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]

1F CLEAR WAVEDATA

Clear the WaveSample data in the source range.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, Layer # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]
0e	Data range start offset (from beginning of
0f	WaveSample data), Data offset = [0..FFFF]
0g	Start Offset = 0efgh (long word, 20 bits)
0h	(see note at top of Section 4.2)
0i	Data range end offset (from beginning of
0j	WaveSample data), Data offset = [0..FFFF]
0k	End Offset = 0ijkl (long word, 20 bits)
0l	(see note at top of Section 4.2)

20 COPY WAVEDATA

Copy the WaveSample data from the source to the destination. The destination WaveSample must already exist.

00	Source instrument number hi byte (always 0)
0a	a = Source instrument number lo byte, # = [0..7]
00	Source layer number hi byte (always 0)
0b	b = Source layer number lo byte, # = [0..7]
0c	c = Source WaveSample number hi byte
0d	d = Source WaveSample number lo byte, # = [1..127]
00	Destination instrument number hi byte (=0)
0e	e = Destination instrument number lo byte
00	Destination layer number hi byte (always 0)
0f	f = Destination layer number lo byte, # = [0..7]
0g	g = Destination WaveSample number hi byte
0h	h = Destination WaveSample number lo byte
0i	Source data range start offset (from
0j	beginning of source WaveSample data),
0k	Data offset = [0..FFFFFF]
0l	Start Offset = 0ijkl (long word, 20 bits)
0m	Source data range end offset (from beginning
0n	of source WaveSample data),
0o	Data offset = [0..FFFFFF]
0p	End Offset = 0mnop (long word, 20 bits)
0q	Destination data range start offset (from
0r	beginning of destination WaveSample data),
0s	Data offset = [0..FFFFFF]
0t	Start Offset = 0qrst (long word, 20 bits)

21 ADD WAVEDATA

Add the source data to the destination and store the result in the destination.

00	Source instrument number hi byte (always 0)
0a	a = Source instrument number lo byte, # = [0..7]
00	Source layer number hi byte (always 0)
0b	b = Source layer number lo byte, # = [0..7]
0c	c = Source WaveSample number hi byte
0d	d = Source WaveSample number lo byte, # = [1..127]
00	Destination instrument number hi byte (=0)
0e	e = Destination instrument number lo byte
00	Destination layer number hi byte (always 0)
0f	f = Destination layer number lo byte, # = [0..7]
0g	g = Destination WaveSample number hi byte
0h	h = Destination WaveSample number lo byte
0i	Source data range start offset (from
0j	beginning of source WaveSample data),
0k	Data offset = [0..FFFFFF]
0l	Start Offset = 0ijkl (long word, 20 bits)
0m	Source data range end offset (from beginning
0n	of source WaveSample data),
0o	Data offset = [0..FFFFFF]
0p	End Offset = 0mnop (long word, 20 bits)
0q	Destination data range start offset (from
0r	beginning of destination WaveSample data),
0s	Data offset = [0..FFFFFF]
0t	Start Offset = 0qrst (long word, 20 bits)
00	Clip prevention flag hi byte, (always 0)
0u	u = Clip prevention flag lo byte, (clear = 00, set = 01)

2.2 SCALE WAVEDATA

Scale the data based on a scaling ramp between the scale factor points.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, Layer # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]
0e	Data range start offset (from beginning of
0f	WaveSample data);
0g	data offset = [0..FFFFFF]
0h	Start Offset = 0efgh (long word, 20 bits)
0i	Data range end offset (from beginning of
0j	WaveSample data);
0k	data offset = [0..FFFFFF]
0l	End Offset = 0ijkl (long word, 20 bits)
00	Scale factor start point. (stored as 2
0m	bytes, hi = 8 bit integer, lo = 7 bit fraction);
0n	factor = mno. (See note at top of Section 4.4.)
0o	Start point range = [0.0..255.99]
00	Scale factor end point. (stored as 2 bytes,
0p	hi = 8 bit integer, lo = 7 bit fraction);
0q	factor = pqr. (See note at top of Section 4.4.)
0r	End point range = [0.0..255.99]
00	Scale ramp depth hi byte (always 0)
0s	s = Scale ramp depth lo byte depth = [0..6] (0=3.0dB, 1=3.5db,..., 6=6.0dB)

2.3 INVERT WAVEDATA

Invert the WaveSample data in the source range, i.e. negate the sample values.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, Layer # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]
0e	Data range start offset (from beginning of
0f	WaveSample data);
0g	data offset = [0..FFFFFF]
0h	Start Offset = 0efgh (long word, 20 bits)
0i	Data range end offset (from beginning of
0j	WaveSample data);
0k	data offset = [0..FFFFFF]
0l	End Offset = 0ijkl (long word, 20 bits)

2.4 REVERSE WAVEDATA

Reverse the WaveSample data in the source range.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, Layer # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]
0e	Data range start offset (from beginning of
0f	WaveSample data);
0g	data offset = [0..FFFF]
0h	Start Offset = 0efgh (long word, 20 bits)
0i	Data range end offset (from beginning of
0j	WaveSample data);
0k	data offset = [0..FFFF]
0l	End Offset = 0ijkl (long word, 20 bits)

2.5 REPLICATE WAVEDATA

Append the WaveSample data in the source range to itself repeatedly until the whole WaveSample data block is filled.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]
0e	Data range start offset (from beginning of
0f	WaveSample data);
0g	data offset = [0..FFFF]
0h	Start Offset = 0efgh (long word, 20 bits)
0i	Data range end offset (from beginning of
0j	WaveSample data);
0k	data offset = [0..FFFF]
0l	End Offset = 0ijkl (long word, 20 bits)

26 CROSS FADE LOOP

Make a cross fade loop of a certain size based on a scaling ramp.

00 Instrument number hi byte (always 0)
 0a a = Instrument number lo byte, # = [0..7]
 00 Layer number hi byte (always 0)
 0b b = Layer number lo byte, # = [0..7]
 0c c = WaveSample number hi byte
 0d d = WaveSample number lo byte, # = [1..127]
 0e Fade zone size (see note at top of section 4.4)
 0f Size = [0..FFFFFF], same as WaveData offset
 0g
 0h Fade zone = 0efgh (long word, 20 bits)
 00 Scale ramp depth hi byte (always 0)
 0i i = Scale ramp depth lo byte depth = [0..6] (0=3.0dB, 1=3.5db,...,
 6=6.0dB)

27 FADE IN WAVEDATA

Fade in the WaveSample data in the source range.

00 Instrument number hi byte (always 0)
 0a a = Instrument number lo byte, # = [0..7]
 00 Layer number hi byte (always 0)
 0b b = Layer number lo byte, # = [0..7]
 0c c = WaveSample number hi byte
 0d d = WaveSample number lo byte, # = [1..127]
 0e Data range start offset (from beginning of
 0f WaveSample data);
 0g data offset = [0..FFFFFF]
 0h Start Offset = 0efgh (long word, 20 bits)
 0i Data range end offset (from beginning of
 0j WaveSample data);
 0k data offset = [0..FFFFFF]
 0l End Offset = 0ijkl (long word, 20 bits)
 00 Scale ramp depth hi byte (always 0)
 0m m = Scale ramp depth lo byte depth = [0..6] (0=3.0dB, 1=3.5db, ...,
 6=6.0dB)

28 FADE OUT WAVEDATA

Fade out the WaveSample data in the source range.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]
0e	Data range start offset (from beginning of
0f	WaveSample data);
0g	data offset = [0..FFFF]
0h	Start Offset = 0efgh (long word, 20 bits)
0i	Data range end offset (from beginning of
0j	WaveSample data);
0k	data offset = [0..FFFF]
0l	End Offset = 0ijkl (long word, 20 bits)
00	Scale ramp depth hi byte (always 0)
0m	m = Scale ramp depth lo byte depth = [0..6] (0=3.0dB, 1=3.5db, ..., 6=6.0dB)

29 REVERSE CROSS FADE LOOP

The whole loop section is faded out, reversed, added to itself, and then a normal cross fade loop is created.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]
0e	Fade zone size (see note at top of Section 4.4)
0f	Size = [0..FFFF], same as WaveData offset
0g	
0h	Fade zone = 0efgh (long word, 20 bits)
00	Scale ramp depth hi byte (always 0)
0i	i = Scale ramp depth lo byte depth = [0..6] (0=3.0dB, 1=3.5db, ..., 6=6.0dB)

2 A ENSEMBLE CROSS FADE LOOP

A normal cross fade loop is created where the cross fade zone equals the loop length.

```

00      Instrument number hi byte (always 0)
0a      a = Instrument number lo byte, # = [0..7]
00      Layer number hi byte (always 0)
0b      b = Layer number lo byte, # = [0..7]
0c      c = WaveSample number hi byte
0d      d = WaveSample number lo byte, # = [1..127]
00      Fade zone size (see note at top of Section 4.4)
00
00      Fade zone = 00000 (long word, 20 bits)
00      Scale ramp depth hi byte (always 0)
0e      e = Scale ramp depth lo byte depth = [0..6] (0=3.0dB, 1=3.5db, ...,
        6=6.0dB)

```

2 B BOWTIE CROSS FADE LOOP

A cross fade loop is created such that the data at each of the loop points is the same. The drawing of the fade lines forms a bowtie shape, hence the name. The cross fade zone length must not be greater than half of the loop length.

```

00      Instrument number hi byte (always 0)
0a      a = Instrument number lo byte, # = [0..7]
00      Layer number hi byte (always 0)
0b      b = Layer number lo byte, # = [0..7]
0c      c = WaveSample number hi byte
0d      d = WaveSample number lo byte, # = [1..127]
0e      Fade zone size (see note at top of Section 4.4)
0f      Size = [0..FFFFFF], same as WaveData offset,
0g
0h      Fade zone = 0efgh (long word, 20 bits)
00      Scale ramp depth hi byte (always 0)
0i      i = Scale ramp depth lo byte depth = [0..6] (0=3.0dB, 1=3.5db, ...,
        6=6.0dB)

```

2C LENGTHEN LOOP

The data at the end of the loop is faded out, reversed, and added such that the faded sections overlap.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]
0e	Fade zone size (see note at top of Section 4.4)
0f	Size = [0..FFFF], same as WaveData offset,
0g	
0h	Fade zone = 0efgh (long word, 20 bits)
00	Scale ramp depth hi byte (always 0)
0i	i = Scale ramp depth lo byte depth = [0..6] (0=3.0dB, 1=3.5db, ..., 6=6.0dB)

2D MIX WAVEDATA

Mix the data of the source WaveSample into the destination WaveSample. If there is any pitch difference between the WaveSamples, the command will fail because there is no way to correct the pitch difference.

00	Source instrument number hi byte (always 0)
0a	a = Source instrument number lo byte, # = [0..7]
00	Source layer number hi byte (always 0)
0b	b = Source layer number lo byte, # = [0..7]
0c	c = Source WaveSample number hi byte
0d	d = Source WaveSample number lo byte, # = [1..127]
00	Destination instrument number hi byte (=0)
0e	e = Destination instrument number lo byte
00	Destination layer number hi byte (always 0)
0f	f = Destination layer number lo byte, # = [0..7]
0g	g = Destination WaveSample number hi byte
0h	h = Destination WaveSample number lo byte
0i	i = Balance control hi byte (always 0)
0j	j = Balance control lo byte, # = [-127..127] (-127 is all source, 0 = equal balance, and 127 is all destination)

0001 = 1

0010 = 2

1101

2

0A

00

8 bits in byte
4 Hex digits
10

256

2E MERGE/SPLICE WAVEDATA

Merge the data of the source WaveSample into the destination WaveSample using a variable cross-fade. A splice is achieved using zero length cross-fade. If there is any pitch difference between the WaveSamples, the command will fail because there is no way to correct the pitch difference.

00	Source instrument number hi byte (always 0)
0a	a = Source instrument number lo byte, # = [0..7]
00	Source layer number hi byte (always 0)
0b	b = Source layer number lo byte, # = [0..7]
0c	c = Source WaveSample number hi byte
0d	d = Source WaveSample number lo byte, # = [1..127]
00	Destination instrument number hi byte (=0)
0e	e = Destination instrument number lo byte
00	Destination layer number hi byte (always 0)
0f	f = Destination layer number lo byte, # = [0..7]
0g	g = Destination WaveSample number hi byte
0h	h = Destination WaveSample number lo byte
0i	Fade zone size [loop size > size < (loop
0j	start - WaveData start)],
0k	Size = [0..FFFFFF], same as WaveData offset
0l	Fade zone = 0ijkl (long word, 20 bits)
00	Scale ramp depth hi byte (always 0)
0m	m = Scale ramp depth lo byte depth = [0..6] (0=3.0dB, 1=3.5db, ... , 6=6.0dB)
0n	n = Balance control hi byte
0o	o = Balance control lo byte, # = [-127..127] (-127 is all source, 0 = equal balance, and 127 is all destination)

2F VOLUME SMOOTHING

The amplitude of a section of data is made the same as the amplitude at the beginning of the section. This is a functions like a digital dynamics compressor.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]
0e	Data range start offset (from beginning of
0f	WaveSample data);
0g	data offset = [0..FFFF]
0h	Start Offset = 0efgh (long word, 20 bits)
0i	Data range end offset (from beginning of
0j	WaveSample data);
0k	data offset = [0..FFFF]
0l	End Offset = 0ijkl (long word, 20 bits)
00	Scale ramp depth hi byte (always 0)
0m	m = Scale ramp depth lo byte depth = [0..6] (0=3.0dB, 1=3.5db, ..., 6=6.0dB)
00	Smoothness parameter hi byte (always 0)
0n	n = Smoothness lo byte, #=[0..4] (0 is very fine, 4 is very coarse)

41 NORMALIZE GAIN

The WaveData is scaled up to the full digital signal range.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]

42 SYNTHESIZED LOOP

Given the current loop points in the specified WaveSample and the smoothness parameter, a loop is generated using an intelligent randomized algorithm.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]
00	Smoothness parameter hi byte (always 0)
0e	e = Smoothness lo byte, #=[0..4] (0 is very fine, 4 is very coarse)

43 BIDIRECTIONAL CROSS FADE LOOP

This is a variation of the normal cross fade for bidirectional loops. The cross fade zone length must not be greater than half of the loop length.

00	Instrument number hi byte (always 0)
0a	a = Instrument number lo byte, # = [0..7]
00	Layer number hi byte (always 0)
0b	b = Layer number lo byte, # = [0..7]
0c	c = WaveSample number hi byte
0d	d = WaveSample number lo byte, # = [1..127]
0e	Fade zone size (see note at top of Section 4.4)
0f	Size = [0..FFFFFF], same as WaveData offset,
0g	
0h	Fade zone = 0efgh (long word, 20 bits)
00	Scale ramp depth hi byte (always 0)
0i	i = Scale ramp depth lo byte depth = [0..6] (0=3.0dB, 1=3.5db, ..., 6=6.0dB)

5 Response Command Status Codes

These codes are the low byte of the data word in a response command (see **Section 4.1**). Status codes are listed in hexadecimal. Each code indicates how the receiver processed the preceding command or if it was able to process it at all. Each status code is sent in 12 bit word format as described in **Section 2.3**. The given meaning is intended as an explanation of the error and possible error recovery actions.

# (hex)	Name	Meaning
00	ACK	Yes, OK, it worked, cool, and it's jammin'.
01	WAIT	The receiver needs extra time to process the command, load an overlay, etc. The transmitter should reset its message timer for 30 seconds. Chill...
02	INSERT SYSTEM DISK	The system disk must be inserted into the EPS-16 PLUS disk drive before the command is executed. The transmitter should prompt the user to insert the disk, and after the user is done, re-transmit the command.
03	INVALID PARAMETER NUMBER	A parameter number in the last GET or PUT parameter command doesn't exist.
04	INVALID PARAMETER VALUE	A parameter value in the last GET or PUT parameter command is out of range.
05	INVALID INSTRUMENT	The instrument(s) specified doesn't exist, i.e. is not loaded.
06	INVALID LAYER	The LAYER(s) specified doesn't exist.
07	LAYER IN USE	The destination LAYER already exists and cannot be used.
08	INVALID WAVESAMPLE	The WaveSample(s) specified doesn't exist.
09	WAVESAMPLE IN USE	The destination WaveSample already exists and cannot be used.
0A	INVALID WAVESAMPLE DATA RANGE	The data range specified doesn't exist within the specified WaveSample.
0B	FILE NOT FOUND	The filename in the command cannot be found in the directory.
0C	MEMORY FULL	There is not enough RAM needed to execute the command. Used when there is pitch difference during the execution of Mix, Merge, or Splice.
0D	INSTRUMENT IN USE	The specified instrument cannot be used for the given command.
0E	NO MORE LAYERS	All eight layers are in use.
0F	NO MORE WAVESAMPLES	All 127 WaveSamples are in use.
10	reserved	Reserved for internal EPS-16 PLUS use.
11	WAVESAMPLE IS A COPY	The WaveData in the specified WaveSample is a copy of data in another WaveSample. The command can only be executed on the WaveSample containing the data.
12	ZONE TOO BIG	The cross fade zone is too big. Make it smaller or move the loop points away from the ends of the WaveSample data.

# (hex)	Name	Meaning
13	SEQUENCER MUST BE STOPPED	The command could not be executed because the sequencer was running. Stop the sequencer and redo the command.
14	DISK ACCESS IN PROGRESS	Current disk activity prevented the execution of the command.
15	DISK FULL	The disk is full and no files can be stored.
16	LOOP IS TOO LONG	The processing command could not be completed because the loop length is too long. This is currently used only by the synthesized loop command.
17	NAK	Something was wrong with the last data transfer which could not be processed.
18	NO LAYER EDIT	When the WaveSample number is zero, a PUT PARAMETER message will effect all WaveSamples on the given layer. This status code is used when this type of editing is not allowed, especially WaveData offset editing.
19	NO MORE PITCH TABLES	Only eight pitch tables can be created within an instrument.
1A	CROSS FADE LENGTH IS ZERO	The cross fade length cannot be zero in this command.
1B	CROSS FADE LENGTH IS GREATER THAN 50%	The cross fade zone must be less than or equal to fifty percent of the loop length.
1C	LOOP START TOO CLOSE TO SAMPLE START	The processing command could not be completed because there was not enough room between sample start and loop start.
1D	LOOP END TOO CLOSE TO SAMPLE END	The processing command could not be completed because there was not enough room between loop end and sample end.
1E	QUIET LAYER	The layer being edited is not in the patch so any parameter changes will not be heard.

6 Button Numbers

The following table lists the valid button number values for the lo byte in the data word of a VIRTUAL BUTTON PRESS command. Button numbers are listed in hexadecimal. Each button number is sent in 12 bit word format as described in Section 2.3. See Section 4.3, Command 40 for the message format.

Logical Button Number (hex)	Front Panel Button Name	Logical Button Number (hex)	Front Panel Button Name
00	Instrument 1	18	Effects
01	Instrument 2	20	Up Arrow (increment)
02	Instrument 3	21	Down Arrow (decrement)
03	Instrument 4	22	Left Arrow (scroll left)
04	Instrument 5	23	Cancel • No
05	Instrument 6	24	Right Arrow (scroll right)
06	Instrument 7	25	Enter • Yes
07	Instrument 8	30	0 Track
10	Sample	31	1 Env1
11	Command Mode	32	2 Env2
12	Effect • Select	33	3 Env3
	• Bypass	34	4 Pitch
13	Edit Mode	35	5 Filter
14	Load Mode	36	6 Amp
15	System • MIDI	37	7 LFO
16	Seq • Song	38	8 Wave
17	Instrument	39	9 Layer

Example: Remotely change Directory with Virtual Button Presses.

(in hexadecimal notation, assuming MIDI BASE CHANNEL=1, transmitted as 00)

Header	Command	Button	Tail	
F0 0F 03 00	40	00 14	F7	press Load button
F0 0F 03 00	40	00 15	F7	press System•MIDI button
F0 0F 03 00	40	00 20	F7	press Up Arrow button
F0 0F 03 00	40	00 25	F7	press Enter•Yes button

NOTE: Each Virtual Button Press command requires its own complete SYSEX packet. A brief delay (approximately 2-300 msecs) between each packet is recommended.

K = 8 H65
Z = 6 H1

7 Parameter Block Data Descriptions

The parameter blocks transmitted in the PUT INSTRUMENT, PUT LAYER, PUT WAVESAMPLE, and PUT PITCH TABLE commands are described here. Word offsets are listed in decimal. See Section 4.2 for detailed descriptions of the PUT commands.

7.1 Instrument Parameter Block

This table describes the data transmitted by the second message of the PUT INSTRUMENT command. All data in the data block is transmitted using the 16 bit word format described in Section 2.3. The table describes the data as it exists in RAM: i.e. the given words offsets are the RAM offsets, not the MIDI data offsets. See Section 9 for the parameter value ranges.

Word Offset (decimal)	Data Word Description (all values in hi byte of word)
00-11	Name (12 ASCII bytes, one byte per word)
12	MIDI Out Channel
13	MIDI Out Program Number
14	Pressure Mode
15	Total Instrument Size in Blocks (1 Block = 256 words)
16	MIDI Status

(Note: the words in the following group each contain bit maps of layers, where bit 0=LYR 1, bit 1=LYR 2, etc. Active layers are indicated by 1 and inactive layers by 0.)

17	Patch 0
18	Patch 1
19	Patch 2
20	Patch 3
21	Key Down Layers
22	Key Up Layers
23	Current Patch Select Mode
24	unused
25	Instrument ID Field (\$FFFF indicates EPS-16 PLUS)
26	Key Range - Lo Key
27	Key Range - Hi Key
28	Transposition - number of semitones (signed)

(Note: the following Offsets are relative to the instrument base address in EPS-16 PLUS RAM.)

29-44	Pitch Table Offsets
45-60	Layer Offsets
61-316	WaveSample Offsets
317	Effect Offset
318-322	unused

7.2 Layer Parameter Block

This table describes the data transmitted by the second message of the PUT LAYER command. All data in the data block is transmitted using the 16 bit word format described in Section 2.3. The table describes the data as it exists in RAM: *i.e. the given words offsets are the RAM offsets, not the MIDI data offsets. Refer to Section 9 for the parameter value ranges.*

p. 42

Word Offset (decimal)	Data Word Description (all values in hi byte of word)
00-11	Name (12 ASCII bytes, one byte per word)
12	- high byte = Glide Mode; - - low byte = Delay Modulation by Velocity Amount
13	- high byte = Glide Time; - low byte = Restrike Decay Time
14	- Legato Layer Number
15	- Velocity Lo
16	- Velocity Hi
17	- Pitch Table Number
18	Delay Time
19-106	Layer Map (one WaveSample number per key in hi byte)

7.3 WaveSample Parameter Block

This table describes the data transmitted by the second message of the PUT WAVESAMPLE command. All data in the data block is transmitted using the 16 bit word format described in Section 2.3. The table describes the data as it exists in RAM: i.e. the given words offsets are the RAM offsets, not the MIDI data offsets. Refer to Section 9 for the parameter value ranges.

Word Offset (decimal)	Data Word Description (all values in hi byte of word unless otherwise specified)
00-11	Name (12 ASCII bytes, one byte per word)
12	WaveSample Copy Number (if non-zero, this is the number of the WaveSample containing the sample data)
13	WaveSample Copy Layer (if non-zero, this is the number of the layer containing the original WaveSample)
14-35	Pitch Envelope #1 (see Section 7.3.1)
36-57	Filter Envelope #2 (see Section 7.3.1)
58-79	Amplitude Envelope #3 (see Section 7.3.1)
80	high byte = Root Key - MIDI key number; low byte = Volume Modulator Crossfade Fadecurve
81	Pitch Envelope Amount
82	LFO Amount
83	Random Modulation Amount
84	Pitch Wheel Bend Range
85	Pitch Modulation Source
86	Fine Tune (signed 7 bit fraction in hi byte)
87	Pitch Modulation Amount
88	Filter Mode
89	FC #1 Cutoff
90	FC #2 Cutoff
91	FC #1 Keyboard Amount
92	FC #2 Keyboard Amount
93	FC #1 Filter Envelope Amount
94	FC #2 Filter Envelope Amount
95	FC #1 Modulation Source
96	FC #2 Modulation Source
97	FC #1 Modulation Amount
98	FC #2 Modulation Amount
99	high byte = Volume; low byte = Output Bus
100	high byte = Volume Modulation Source; low byte = Pan Modulation Source

Word Offset (decimal)	Data Word Description (all values in hi byte of word)
101	Volume Modulator Crossfade-In Breakpoint A
102	Volume Modulator Crossfade-In Breakpoint B
103	Volume Modulator Crossfade-Out Breakpoint C
104	Volume Modulator Crossfade-Out Breakpoint D
105	high byte = unused; low byte = Pan Position
106	high byte = Volume Modulation Amount; low byte = Pan Modulation Amount
107	high byte = LFO Waveform; low byte = Boost Switch
108	high byte = LFO Speed; low byte = LFO Rate Modulation Amount
109	LFO Depth
110	high byte = LFO Delay Time; low byte = LFO Depth Modulation Amount
111	high byte = LFO Depth Modulation Source; low byte = LFO Rate Modulation Source
112	LFO Mode
113	Random Modulator Frequency
114	Loop Mode
115-118	Sample Start Offset (left justified 32 bit field using hi bytes of each word - shift right 9 for word offset)
119-122	Sample End Offset (left justified 32 bit field using hi bytes of each word - shift right 9 for word offset)
123-126	Loop Start Offset (left justified 32 bit field using hi bytes of each word - shift right 9 for word offset)
127-130	Loop End Offset (left justified 32 bit field using hi bytes of each word - shift right 5 for 4 bit loop end fraction)
131	Sample Rate (sample period = rate * 1.6 microseconds)
132	Key Range - Lo Key
133	Key Range - Hi Key
134	Start or Loop Modulation Source
135	Start or Loop Modulation Amount
136	Start or Loop Modulation Range
137	Wave Modulation Type
138	unused

7.3.1 WaveSample Envelope Description

This table describes the structure of each of the three envelopes in each WaveSample. All offsets are given from the base of the envelope: e.g. the base of the filter envelope is offset 36 words from the start of the data block in a PUT WAVESAMPLE command. Refer to **Section 9** for the parameter value ranges.

Word Offset (decimal)	Data Word Description (all values in hi byte of word)
0	Envelope Type (default envelopes)
1	Soft Level 0 - initial level
2	Hard Level 0 - initial level
3	Time 1 - attack time; time from initial level to level 1
4	Soft Level 1 - peak level
5	Hard Level 1 - peak level
6	Time 2 - first decay time; time from level 1 to level 2
7	Soft Level 2
8	Hard Level 2
9	Time 3 - second decay; time from level 2 to level 3
10	Soft Level 3
11	Hard Level 3
12	Time 4 - third decay; time from level 3 to level 4
13	Soft Level 4 - sustain
14	Hard Level 4 - sustain
15	Time 5 - release time; time from level 4 to level 5
16	Soft Velocity Curve Mode
17	Level 5 - release breakpoint relative to sustain level (+/-)
18	Time 6 - second release time; time from level 5 to 0
19	Time 1 velocity sensitivity
20	Keyboard Time Scaling
21	Envelope Mode (0=normal, 1=cycle, 2=repeat)

7.4 Pitch Table Block

This table describes the data transmitted by the second message of the PUT PITCH TABLE command. All data in the data block is transmitted using the 16 bit word format described in **Section 2.3**. The table describes the data as it exists in RAM: i.e. the words offsets given are the RAM offsets, not the MIDI data offsets.

Word Offset (decimal)	Data Word Description (all values in hi byte of word)
00-11	Name (12 ASCII bytes, one byte per word)
12-99	Key Map (88 key values, one key per word; for each word, bits 15:9=semitone, and bits 8:3=semitone fraction)
100-106	unused

8 Examples of Data Transfer Protocol

Example #1: A WaveData Transfer

Assume that EXT wants to receive a portion of WaveSample data from the EPS-16 PLUS. This is one of the most common data transfers, and the following steps describe the commands sent to and from the EPS-16 PLUS to affect the data transfer. Note that the basic steps outlined below apply to any instance of data transfer between EXT and the EPS-16 PLUS.

- 1.) to EPS-16 PLUS *from* EXT:
GET WAVESAMPLE DATA: specify the instrument, layer, WaveSample, and the start and end offsets.
- 2.) to EXT *from* EPS-16 PLUS:
RESPONSE: if all the parameters of the GET WAVESAMPLE DATA command were OK, then the ACK status code is sent (followed by the PUT command described below); if any part of the GET command was erroneous, an error status code is sent and the data transfer is complete.

PUT WAVESAMPLE DATA (first part): the PUT command is sent only if the RESPONSE above was an ACK; the parameters and values here are the same as those sent in the GET WAVESAMPLE DATA command in step 1.
- 3.) to EPS-16 PLUS *from* EXT:
RESPONSE: if EXT still wants the data, the ACK status code should be sent before the EPS-16 PLUS times out (within 2 seconds); otherwise the data transfer is complete.
- 4.) to EXT *from* EPS-16 PLUS:
PUT WAVESAMPLE DATA (second part): all the samples within the data range given in the GET WAVESAMPLE DATA are transmitted using the 16 bit word format described in **Section 2.3**. The block of samples forms the <Message> within the system exclusive packet frame described in **Section 2**.
- 5.) to EPS-16 PLUS *from* EXT:
RESPONSE: if the data was successfully received, the ACK status code should be sent. If an error occurred during the second part of the PUT WAVESAMPLE DATA command, the data transfer must be started again from step 1, since the EPS-16 PLUS will not re-transmit the second part of the PUT WAVESAMPLE DATA again regardless of the status code.

Example #2: A PUT PARAMETER Message

Assume that EXT wants to edit the Filter 1 Cutoff value for WaveSample 1 in Instrument 1. (in hexadecimal notation, assuming MIDI BASE CHANNEL=1, transmitted as 00)

to EPS-16 PLUS *from* EXT:

Header	Cmd	Inst / Lyr / WS	filter 1 cutoff	value=5	Tail
F0 0F 03 00	11	00 00 00 00 00 01	14 01 00 00 00	05	F7

9 Parameter Numbers

The following table of parameter numbers is used in the GET and PUT PARAMETER commands. See **Section 4.2** for the command message format. The complete parameter number word consists of two bytes: the page number (high byte) and the item number (low byte). The parameter number is transmitted using the 12 bit word format described in **Section 2.3**. All parameter values, including the WaveData offsets, are right justified within a 24 bit binary word and then transmitted as two 12 bit words according to **Section 2.3**. See **Section 8** for an example of a PUT PARAMETER message.

NOTE 1: Parameters marked as "receive only" do not transmit SysEx when edited from the EPS-16 PLUS front panel. The EPS-16 PLUS must receive a GET PARAMETER command in order for it to transmit the current values of these parameters.

NOTE 2: Some functions, such as those on the Sample and Effect Select•Bypass pages, neither transmit nor receive PUT or GET PARAMETER commands.

NOTE 3: Parameters marked with a "*" do not receive single PUT PARAMETER commands. Parameters marked with a "***" are not available for use with single GET and PUT PARAMETER commands. The values for both of these parameter groups can be transmitted to or retrieved from the EPS-16 PLUS by using the PUT or GET INSTRUMENT, LAYER and WAVESAMPLE commands.

9.1 SYSTEM•MIDI PARAMETERS

NOTE: If a System•MIDI parameter is being PUT, the instrument, layer, and WaveSample values are irrelevant, but still must be included in the message; it is recommended that they be transmitted as all zeros. The MIDI Transmit Mode cannot be changed with a PUT command because it is used when commands are being processed.

SysEx High Byte (hex): 34

	Low Byte (hex)	Range (decimal)
Free System Blocks	00	0-10000 (Read Only)
Free Disk Blocks	0A	0-10000 (Read Only)
Master Tune	01	-99 to +99
Global Bend Range	0B	0-12
Touch Sensitivity Curve (kbd only)	0C	0-15 (Table 1) **
All Notes Off Enable (rack only)	0C	0-1 (OFF-ON) **
Foot Pedal Mode	02	0-1 (Volume-Mod Pedal) *
Sustain Foot Switch Mode	0D	0-1 (Sustain-R Patch Sel) *
Aux Foot Switch Mode	0E	0-2 (Off-Start/Stop-L Patch Sel) *
Auto-Loop Finding Enable	03	0-1 (OFF-ON) *
FX Send Bus2 Enable	n/a	0-1 (OFF-ON) **
FX Send Bus3 Enable	n/a	0-1 (OFF-ON) **
Midi Base Channel	05	0-15 (Channels 1-16) *
Midi Transmit Mode	06	0-1 (Base or Inst Channels) *
Base Channel Pressure Mode	0A	0-2 (OFF-KEY-CHANNEL) **
Midi In Mode	07	0-4 (Table 9) *
Midi Controllers Enable	0B	0-1 (OFF-ON) *
Midi SYSEX Enable	0C	0-1 (OFF-ON; only sends ON value)
Midi Program Change Enable	08	0-1 (OFF-ON) *
Midi Song Select Enable	0D	0-1 (OFF-ON)
Midi XCTRL Controller Number	09	0-127
Multi Controllers Enable	0E	0-1 (OFF-ON) **

9.2 SEQUENCE•SONG PARAMETERS

SysEx High Byte (hex): 2C

	Low Byte (hex)	Range (decimal)
Tempo	02	20-250 (bpm)
Loop Enable	03	0-1 (OFF-ON) *
Clock Source	04	0-1 (INTERNAL-MIDI) *
Click Mode	05	0-2 (OFF-ON-REC) **
Click Note Value	0B	0-9 (Table 13)
Click Volume	06	0-99
Click Pan	07	-99 to +99
Click Output Bus	0E	0-3 (BUS1-BUS2-BUS3-AUX1) *
Seq Countoff Mode	08	0-3 (OFF-ON-RECORD-QUIET) *
Record Mode	09	0-2 (REPLACE-ADD-LOOPED) *
Record Source	0D	0-3 (BOTH-KYBD-MIDI-MULTI) *

9.2.1 TRACK PARAMETERS

SysEx High Byte (hex): 00

	Low Byte (hex)	Range (decimal)
Sequence Goto Bar	00	0-999 *
Song Goto Step	01	0-99 *
Sequence Track Status	00	0-2 (Mute-Play-Solo) *
Song Track Status	03	0-2 (Mute-Play-Solo) *

(NOTE: For Track Status values described above, the Instrument number lo byte determines which Track's Status will be edited. See Section 4.2 for the GET and PUT command message format.)

Track Mix	01	0-99
Track Pan	02	-100 to +99 (-100=WS*)
Track Output Bus	04	0-5 (WAVESAMPLE-BUS1-BUS2-BUS3-AUX1-ROTATE)
Effect Control Enable	05	0-1 (OFF-ON)
Multi-In Midi Channel	06	0-15 (Channels 1-16)

9.3 INSTRUMENT PARAMETERS

SysEx High Byte (hex): 28

	Low Byte (hex)	Range (decimal)
Patch (layers active in current patch)	00	0-255
Key Down Layers	01	0-255
Key Up Layers	02	0-255
(NOTE: The above values select the layers in each parameter. Each layer is represented by a bit; if the bit is set, that layer will sound.)		
Current Patch Select Mode	09	0-5 (LIVE,00,0*,*0,**,HELD) *
Midi Out Channel	03	0-15 (Channels 1-16)
Midi Out Program	04	0-127
Pressure Mode	05	0-2 (OFF-KEY-CHANNEL) **
Instrument Size	07	0-10000 (Read Only)
Midi Status	06	0-3 (BOTH-LOCAL-MIDI-EXT) *
Range Low Key	0A	0-127 (*=value out of range)
Range High Key	0B	0-127 (*=value out of range)
Transpose Amount-Octave	0C	-4 to +4
Transpose Amount-Semitone	0D	-11 to +11
Instrument Name	08	**

9.4 LAYER PARAMETERS

SysEx High Byte (hex): 24

	Low Byte (hex)	Range (decimal)
Glide Mode	00	0-4 (Table 10) * - p. 53
Glide Time	01	0-99
Legato Layer	02	0-7
Velocity Low	03	0-127
Velocity High	0A	0-127
Pitch Table	04	0-9 (STAND,NO PITCH,1-8) **
Delay Time	06	0-5000 (milliseconds)
Delay Mod by Velocity Amount	07	-99 to +99
Restrike Decay Time	08	0-99 (milliseconds)
Layer Name	05	**

9.5 WAVESAMPLE PARAMETERS

SysEx High Byte (hex): 20

	Low Byte (hex)	Range (decimal)
Loop Mode	00	0-4 (Table 8) *
Loop Mod Type	06	0-7 (Table 5) *
Loop Mod Source	07	0-15 (Table 2) *
Loop Mod Amount	08	-99 to +99
Loop Mod Range	09	0-21 (Table 7) *
Sample Start (absolute)	15	0-FFFFFF (hex)
Sample End (absolute)	16	0-FFFFFF (hex)
Loop Start (absolute)	17	0-FFFFFF (hex)
Loop End (absolute)	18	0-FFFFFF (hex)
Loop End Fractional (fine tune)	0A	0-127
WS Key Range Lo (receive only)	0B	0-127
WS Key Range Hi (receive only)	0C	0-127
Loop Position (absolute)	19	0-FFFFFF (hex)
Sample Rate (receive only)	0D	0-127
Sample Start (%)	01	0-99 *
Sample End (%)	02	0-99 *
Loop Start (%)	03	0-99 *
Loop End (%)	04	0-99 *
Loop Position (%)	05	0-99 *

9.6 ENVELOPE PARAMETERS

SysEx High Byte (hex): 04, 08, 0C (ENV1 = 04, ENV2 = 08, ENV3 = 0C)

	Low Byte (hex)	Range (decimal)
Envelope Type	00	0-15 (Table 3)
Level 0 Hard	01	0-99
Level 0 Soft	02	0-99
Level 1 Hard	0B	0-99
Level 1 Soft	0F	0-99
Level 2 Hard	0C	0-99
Level 2 Soft	10	0-99
Level 3 Hard	0D	0-99
Level 3 Soft	11	0-99
Level 4 Hard	0E	0-99
Level 4 Soft	12	0-99
Time 1	03	0-99
Time 2	13	0-99
Time 3	14	0-99
Time 4	15	0-99
Time 5	16	0-99
2nd Release Time	04	0-99
2nd Release Level	17	-99 to +99
Attack Time Mod by Velocity Amt	05	0-99
Keyboard Env Time Scaling	06	0-99
Soft Velocity Curve Mode	07	0-3 (OFF-VEL-VEL1-VEL2)**
Envelope Mode	08	0-2 (NORMAL-CYCLE-REPEAT) *

9.7 PITCH PARAMETERS

SysEx High Byte (hex): 10

	<i>Low Byte (hex)</i>	<i>Range (decimal)</i>
Root Key	01	0-127 (*=value out of range)
Fine Tune	0A	-99 to +99
LFO Amount	02	-15.7 to +15.7 (in 0.1 increments)
Envelope 1 Amount	03	-15.7 to +15.7 (in 0.1 increments)
Random Frequency	05	0-99
Random Pitch Mod Amount	0C	-99 to +99
Bend Range	06	0-13 (13=global) *
Pitch Modulation Source	07	0-15 (Table 2)
Pitch Modulation Amount	0B	-99 to +99
WS Key Range Lo	0B	0-127 (*=value out of range) *
WS Key Range Hi	0C	0-127 (*=value out of range) *

9.8 FILTER PARAMETERS

SysEx High Byte (hex): 14

	<i>Low Byte (hex)</i>	<i>Range (decimal)</i>
Filter Mode	00	0-3 (Table 4) *
Filter 1 Cutoff	01	0-150
Filter 2 Cutoff	0B	0-150
Filter 1 Envelope 2 Amount	02	-99 to +99
Filter 2 Envelope 2 Amount	0C	-99 to +99
Filter 1 Keyboard Amount	03	-99 to +99
Filter 2 Keyboard Amount	0D	-99 to +99
Filter 1 Modulation Source	07	0-15 (Table 2) **
Filter 1 Modulation Amount	0E	-99 to +99
Filter 2 Modulation Source	08	0-15 (Table 2) **
Filter 2 Modulation Amount	0F	-99 to +99

9.9 AMP (VOLUME) PARAMETERS

SysEx High Byte (hex): 18

	<i>Low Byte (hex)</i>	<i>Range (decimal)</i>
WaveSample Volume	01	0-99
Pan	02	-99 to +99
Pan Modulation Source	08	0-15 (Table 2) *
Pan Modulation Amount	0D	-99 to +99
Vol Mod Crossfade-In Point A	03	0-127
Vol Mod Crossfade-In Point B	0B	0-127
Vol Mod Crossfade-Out Point C	04	0-127
Vol Mod Crossfade-Out Point D	0C	0-127
Volume Modulation Source	07	0-15 (Table 2) *
Volume Modulation Amount	0A	0-99
Vol Mod Crossfade Fadecurve	05	0-1 (CROSSFADE-LINEAR) *
Boost	06	0-1 (OFF-ON) *
Output Bus	09	0-4 (BUS1-BUS2-BUS3-AUX1-ROTATE) *

9.10 LFO PARAMETERS

SysEx High Byte (hex): 1C

	<i>Low Byte (hex)</i>	<i>Range (decimal)</i>
LFO Wave	01	0-6 (Table 6) **
LFO Speed	02	0-99
LFO Depth	03	0-99
LFO Delay	04	0-99
LFO Mode	05	0-2 (Reset Off-Reset On-Human) **
LFO Depth Modulation Source	07	0-15 (Table 2) **
LFO Depth Modulation Amount	0A	-99 to +99
LFO Rate Modulation Source	08	0-15 (Table 2) **
LFO Rate Modulation Amount	0B	-99 to +99

9.11 EDIT CONTEXT PARAMETERS

SysEx High Byte (hex): 38

	<i>Low Byte (hex)</i>	<i>Range (decimal)</i>
Current Edit Instr. (receive only)	00	0-7 (Instruments 1-8)
Current Edit Layer	01	0-7 (Layers 1-8) **
Current Edit WaveSample	02	0-128 (0=ALL) **

9.12 EFFECTS PARAMETERS

(NOTE: The FX Algorithm Select parameter on the Effect Select•Bypass screen does not transmit SysEx when edited from the EPS-16 PLUS front panel.)

SysEx High Byte (hex): 30

	Low Byte (hex)	Range (decimal)
<u>ROM-01 HALL REVERB</u>		
Variation	00	0-3 (Variations 1-4)
Reverb Mix BUS1	01	0-99
Reverb Mix BUS2	02	0-99
Reverb Mix BUS3	02	0-99 *
Decay Time	03	0-99
Low Frequency Decay	04	-99 to +99
Diffusion	05	0-99
Hi Frequency Damping	06	0-99
Hi Frequency Bandwidth	07	0-99
Detune Rate	08	0-99
Detune Amount	00	0-99 *
Pre-Delay Time	00	0-500 (milliseconds) **
BUS1 MixMod Source	09	0-10 (Table 11)
BUS1 MixMod Amount	00	-99 to +99 *
<u>ROM-02 44 kHz REVERB</u>		
Variation	00	0-3 (Variations 1-4)
Reverb Mix BUS1	01	0-99
Reverb Mix BUS2	02	0-99
Reverb Mix BUS3	02	0-99 *
Decay Time	03	0-99
Low Frequency Decay	04	-99 to +99
Diffusion	05	0-99
Hi Frequency Damping	06	0-99
Hi Frequency Bandwidth	07	0-99
Detune Rate	08	0-99
Detune Amount	00	0-99 *
Pre-Delay Time	00	0-340 (milliseconds) **
BUS1 MixMod Source	09	0-10 (Table 11)
BUS1 MixMod Amount	00	-99 to +99 *

ROM-03 ROOM REVERB

Variation	00	0-3 (Variations 1-4)
Reverb Mix BUS1	01	0-99
Reverb Mix BUS2	02	0-99
Reverb Mix BUS3	02	0-99 *
Decay Time	03	0-99
Low Frequency Decay	04	-99 to +99
Diffusion	05	0-99
Hi Frequency Damping	06	0-99
Hi Frequency Bandwidth	07	0-99
Detune Rate	08	0-99
Detune Amount	00	0-99 *
Pre-Delay Time	00	0-500 (milliseconds) **
BUS1 MixMod Source	09	0-10 (Table 11)
BUS1 MixMod Amount	00	-99 to +99 *

ROM-04 DUAL DELAYS

Variation	00	0-3 (Variations 1-4)
BUS1 Delay Mix	01	0-99
BUS1 Feedback	02	0-99
BUS1 Time Left	03	0-2000 (milliseconds) **
BUS1 Time Right	00	0-2000 (milliseconds) **
BUS2 Delay Mix	04	0-99
BUS2 Feedback	05	0-99
BUS2 Time Left	06	0-150 (milliseconds) **
BUS2 Time Right	00	0-150 (milliseconds) **

ROM-05 44 kHz DELAYS

Variation	00	0-3 (Variations 1-4)
BUS1 Delay Mix	01	0-99
BUS1 Feedback	02	0-99
BUS1 Time Left	03	0-1300 (milliseconds) **
BUS1 Time Right	00	0-1300 (milliseconds) **
BUS2 Delay Mix	04	0-99
BUS2 Feedback	05	0-99
BUS2 Time Left	06	0-150 (milliseconds) **
BUS2 Time Right	00	0-150 (milliseconds) **

ROM-06 CHORUS+REVERB

Variation	00	0-3 (Variations 1-4)
BUS1 Reverb After Chorus	01	0-99
BUS2 Reverb Mix	02	0-99
Reverb Decay Time	03	0-99
Low Frequency Decay	04	-99 to +99
Diffusion	05	0-99
Hi Frequency Damping	06	0-99
Hi Frequency Bandwidth	06	0-99 *
Reverb Detune Rate	06	0-99 *
Reverb Detune Amount	00	0-99 *
Early Reflection Level	00	0-99 *
Chorus Rate	07	0-99
Chorus Amount	00	0-99 *
Chorus Mix	08	0-99
Chorus Feedback	09	-99 to +99
Chorus MixMod Source	00	0-10 (Table 11) *
Chorus MixMod Amount	00	-99 to +99 *

ROM-07 PHASER+REVERB

Variation	00	0-3 (Variations 1-4)
BUS1 Reverb Mix	01	0-99
BUS2 Reverb Mix	02	0-99
Reverb Decay Time	03	0-99
Low Frequency Decay	03	-99 to +99 *
Diffusion	04	0-99
Hi Frequency Damping	05	0-99
Hi Frequency Bandwidth	05	0-99 *
Reverb Detune Rate	06	0-99
Reverb Detune Amount	00	0-99 *
Early Reflection Level	00	0-99 *
Phase Center	07	0-99
Phase Rate	08	0-99
Phase Width	09	0-99
Phase Feedback	09	-99 to +99 *
Phase Depth	08	0-99 *
Feedback Mod Source	00	0-10 (Table 11) *
Feedback Mod Amount	00	-99 to +99 *

ROM-08 FLANGER+REVERB

Variation	00	0-3 (Variations 1-4)
BUS1 Reverb Mix	01	0-99
BUS2 Reverb Mix	02	0-99
Reverb Decay Time	03	0-99
Low Frequency Decay	04	-99 to +99
Diffusion	05	0-99
Hi Frequency Damping	06	0-99
Hi Frequency Bandwidth	06	0-99 *
Reverb Detune Rate	06	0-99 *
Reverb Detune Amount	00	0-99 *
Early Reflection Level	00	0-99 *
Flange Center	07	0-99
Flange Rate	08	0-99
Flange Width	09	0-99
Flange Feedback	09	-99 to +99 *
Flange Depth	08	0-99 *
Feedback Mod Source	00	0-10 (Table 11) *
Feedback Mod Amount	00	-99 to +99 *

ROM-09 ROTARY SPEAKER+REVERB

Variation	00	0-3 (Variations 1-4)
BUS1 Reverb After Roto	01	0-99
BUS2 Reverb Mix	02	0-99
Reverb Decay Time	03	0-99
Hi Frequency Damping	04	0-99
Diffusion	05	0-99
Rotor Fast	06	0-99
Rotor Slow	00	0-99 *
Rotor Inertia	00	0-99 *
Fast-Slow Switch	07	0-7 (Table 12)
Distortion Gain In	08	0-99
Distortion Gain Out	00	0-99 *
Speaker Gain	00	0-99 *
Distortion Filter	09	0-99

ROM-10 CHOR+REV+DDL

Variation	00	0-3 (Variations 1-4)
BUS1 Reverb After Chorus	01	0-99
BUS2 Reverb Mix	02	0-99
Reverb Decay Time	03	0-99
Low Frequency Decay	04	-99 to +99
Hi Frequency Damping	05	0-99
Reverb Detune Rate	06	0-99
Reverb Detune Amount	00	0-99 *
Chorus Rate	07	0-99
Chorus Amount	00	0-99 *
Chorus Mix	00	0-99 *
Chorus Feedback	00	-99 to +99 *
BUS3 DDL Mix	08	0-99
DDL Time Left	00	0-1500 (milliseconds) **
DDL Time Right	00	0-1500 (milliseconds) **
DDL Feedback	09	0-99
DDL Feedback Mod Source	00	0-10 (Table 11) *
DDL Feedback Mod Amount	00	-99 to +99 *

ROM-11 CMP+DIST+REV

Variation	00	0-3 (Variations 1-4)
BUS1 Reverb After Distortion	01	0-99
BUS2 Reverb Mix	02	0-99
Reverb Decay Time	03	0-99
Hi Frequency Damping	04	0-99
Compressor Threshold	07	0-99
Distortion Gain In	05	0-99
Distortion Gain Out	00	0-99 *
Distortion LPF	06	0-99
Distortion HPF	00	0-99 *
Distortion Flange Rate	08	0-99
System Feedback	09	-99 to +99
Feedback Mod Source	00	0-10 (Table 11) *
Feedback Mod Amount	00	-99 to +99 *

ROM-12 DIST+CHO+REV

Variation	00	0-3 (Variations 1-4)
Distortion into Chorus Mix	01	0-99
Chorus into Reverb Mix	02	0-99
BUS3 Reverb Mix	02	0-99 *
Distortion Gain In	04	0-99
Distortion Gain Out	00	0-99 *
Distortion Filter FC (Cutoff)	05	0-99
Distortion Filter Q (Resonance)	00	0-24 (values 1-25) *
Distortion Mix	00	0-99 *
Reverb Decay Time	03	0-99
Low Frequency Decay	00	-99 to +99 *
Hi Frequency Damping	08	0-99
Hi Frequency Bandwidth	00	0-99 *
Reverb Detune Rate	00	0-99 *
Reverb Detune Amount	00	0-99 *
Early Reflection Level	00	0-99 *
Chorus Rate	06	0-99
Chorus Amount	00	0-99 *
Chorus Mix	07	0-99
Filter Mod Source	09	0-10 (Table 11)
Filter Mod Amount	00	-99 to +99 *

ROM-13 WAH+DIST+REV

Variation	00	0-3 (Variations 1-4)
BUS1 Reverb After Distortion	01	0-99
BUS2 Reverb Mix	02	0-99
Reverb Decay Time	03	0-99
Reverb Damping	04	0-99
Compressor Threshold	07	0-99
Distortion Gain In	05	0-99
Distortion Gain Out	00	0-99 *
Wah Center	06	0-99
Wah Range	00	-99 to +99 *
System Feedback	09	-99 to +99
Range Mod Source	00	0-10 (Table 11) *
Range Mod Amount	00	-99 to +99 *

10 PARAMETER VALUE TABLES (in hexadecimal)**TABLE 1***(hex)*

00	=	SOFT 1
01	=	SOFT 2
02	=	SOFT 3
03	=	SOFT 4
04	=	MEDIUM 1
05	=	MEDIUM 2
06	=	MEDIUM 3
07	=	MEDIUM 4
08	=	FIRM 1
09	=	FIRM 2
0A	=	FIRM 3
0B	=	FIRM 4
0C	=	HARD 1
0D	=	HARD 2
0E	=	HARD 3
0F	=	HARD 4

TABLE 2*(hex)*

00	=	LFO
01	=	RANDM
02	=	ENV1
03	=	ENV2
04	=	PR+VL
05	=	VEL
06	=	VEL 1
07	=	VEL 2
08	=	KBD
09	=	PITCH
0A	=	WHEEL
0B	=	PEDAL
0C	=	XCTRL
0D	=	PRESS
0E	=	WL+PR
0F	=	OFF

TABLE 3*(hex)*

00	=	CURRENT VALUE
01	=	FULL ON
02	=	ALL ZEROS
03	=	FULL VELRANGE
04	=	SLOW STRING
05	=	PIANO DECAY
06	=	PERCUSSION
07	=	RAMP UP
08	=	RAMP DOWN
09	=	SHORT BLIP
0A	=	BRASS FILTER
0B	=	REPEAT TRIANG
0C	=	REPEAT RAMP
0D	=	WIND PITCH
0E	=	REVERB
0F	=	SAVED

TABLE 4*(hex)*

00	=	F1=2/LP F2=2/HP
01	=	F1=3/LP F2=1/HP
02	=	F1=2/LP F2=2/LP
03	=	F1=3/LP F2=1/LP

TABLE 5*(hex)*

00	=	OFF
01	=	LOOP POS
02	=	START
03	=	START+LP
04	=	LOOPSTRT
05	=	LOOP END
06	=	LPSTRT-X
07	=	TRANSWAV

TABLE 6*(hex)*

00	=	TRIANGLE
01	=	SIN/TRIANGLE
02	=	SIN WAVE
03	=	POS/TRIANGLE
04	=	POS/SIN WAVE
05	=	SAWTOOTH
06	=	SQUARE

TABLE 7*(hex)*

00	=	2 MG
01	=	1 MG
02	=	512 K
03	=	256 K
04	=	128 K
05	=	64 K
06	=	32 K
07	=	16 K
08	=	8 K
09	=	4 K
0A	=	2 K
0B	=	1 K
0C	=	512 B
0D	=	256 B
0E	=	128 B
0F	=	64 B
10	=	32 B
11	=	16 B
12	=	8 B
13	=	4 B
14	=	2 B
15	=	1 B

TABLE 8*(hex)*

00	=	FORWARD-NO LOOP
01	=	BACKWARD-NO LOOP
02	=	LOOP FORWARD
03	=	LOOP BIDIRECTION
04	=	LOOP AND RELEASE

TABLE 9*(hex)*

00	=	OMNI
01	=	POLY
02	=	MULTI
03	=	MONO A
04	=	MONO B

TABLE 10*(hex)*

00	=	OFF
01	=	LEGATO
02	=	PEDAL
03	=	TRIGGER
04	=	MINIMODE

TABLE 11*(hex)*

00	=	WHEEL
01	=	PRESS
02	=	PEDAL
03	=	XCTRL
04	=	KBD
05	=	VEL
06	=	KEYDN
07	=	SUSTN
08	=	FTSW2
09	=	PATCH
0A	=	OFF

TABLE 12*(hex)*

00	=	WHEEL
01	=	PRESS
02	=	PEDAL
03	=	XCTRL
04	=	SUSTN
05	=	FTSW2
06	=	PATCH
07	=	OFF

TABLE 13*(hex)*

00	=	1/2
01	=	1/2T
02	=	1/4
03	=	1/4T
04	=	1/8
05	=	1/8T
06	=	1/16
07	=	1/16T
08	=	1/32
09	=	1/32T

Appendix A: EPS-16 PLUS SCSI Data Transfer Protocol

When SCSI is connected between the EPS-16 PLUS and an external device (EXT), transfers of parameters and WaveData can be accelerated using the higher bandwidth of SCSI. The following section describes the sequence of commands and data exchanged between the EPS-16 PLUS and an external device to be used when sending data to the EPS-16 PLUS or getting data from the EPS-16 PLUS. In all transfer sequences, the external device is the SCSI initiator. The external device always starts an SCSI transfer. Because of this, the protocol is different depending on which way the data is going.

Note: In each of the protocol descriptions, **BOLD** is used to indicate MIDI transfers and *Italics* are used to indicate SCSI transfers.

Sending Data to the EPS-16 PLUS

<u>External Device</u>	<u>EPS-16 PLUS</u>	<u>Comments</u>
PUT <cmd>	ACK	After ACK, EPS-16 PLUS waits for SCSI write.
<i>Write</i> if no SCSI, PUT data	ACK	It is also simultaneously waiting for the data via MIDI. If the external device can't establish SCSI communications, then the data is sent via MIDI.

Getting Data from the EPS-16 PLUS

<u>External Device</u>	<u>EPS-16 PLUS</u>	<u>Comments</u>
GET <cmd>	ACK	
ACK	PUT <cmd>	
<i>Read</i>	wait 1 sec...	After second ACK, external device sends SCSI read. If EPS-16 PLUS doesn't receive SCSI read within 1 sec after transmitting the PUT, the data is sent via MIDI; otherwise, the data is sent via SCSI.
	If timeout, PUT data else <i>Write</i>	
ACK		

SCSI Commands Used

The SCSI messages should be any valid SCSI command and the byte transfer mode should be used. The parameters within the SCSI command are irrelevant and should not be used for the actual data addresses of the data. The MIDI PUT command contains the actual parameters of the data. The EPS-16 PLUS only expects to receive some SCSI command in order to transmit or receive data on the SCSI buss. A good rule is to use a SCSI read command when getting data from the EPS-16 PLUS and the SCSI write command when sending data to the EPS-16 PLUS.

The read and write commands described above are the high level SCSI commands and are not intended as substitutes for low level SCSI handshaking, such as the select command, which is necessary for proper data transfer. Refer to the SCSI specification for more details.

Appendix B: A Quick Description of Instrument Format.

NOTE: Appendix B provides a concise overview of Original EPS and EPS-16 PLUS data formats, for use by experienced computer programmers. All of the material contained in this section is documented in greater detail in the body of the External Command Specification.

OVERVIEW:

Assume that an instrument is loaded into a contiguous block of RAM which is 16 bits wide. Assume Motorola (hi byte first) format, two's complement. Notice that the low 3 bits of the 16 bit word are always read as zeros in the Original EPS. This accounts for the funny way things are packed.

The inst, the layer, the WaveSample, the pitchtable, and the effect are each data objects inside the instrument. Each data object has a header of fixed size. Objects are 16 byte chunky.

The first object in an instrument is always the inst. The inst is a data structure which contains some parameters and also some pointers. The pointers are actually offsets from the base address of the inst. These offsets give the location of all other objects in the instrument. These offsets are packed.

OFFSET PACKING:

A packed offset is 32 bits.

The unpacked offset is also 32 bit number, to be used as a byte count.

An offset is packed in memory as two words, or 4 bytes.

packed format:

byte0=aaaaaaaa	byte1=bbbbbbbb
byte2=cccccccc	byte3=ddddEEEE

The hi nibble of the low byte is appended to byte 0 and byte 2 as follows:
 dddd aaaaaaaaa ccccccc

Four zeros are added to the lo part (enforcing the 16 byte chunkiness)
 dddd aaaaaaaaa ccccccc 0000

For completeness, we specify that the high 8 bits of a 32 bit offset are 0.

The final unpacked offset looks like this:
 00000000 ddddaaaa aaaacccc cccc0000

For example:

An unpacked offset value of (hex) \$00123450
 is packed as: \$2300 \$4510

WAVESAMPLE STRUCTURE:

The data in a WaveSample follows immediately after the ws data structure, which is \$120 bytes long.

```
*****
* instrument definition      (parameter numbers in parentheses)
*****
```

Below are the data structure definitions for the inst, layer, and WaveSample.

- * ds.l means declare storage long, which means 4 bytes.
- * ds means ds.w which means word, or 2 bytes.
- * ds.b means one byte.

```
aseg
org 0
```

```
inst_block_size      ds.l 1      * mmu controls these two long words
inst_self_ptr        ds.l 1      * (used for relocation) ptr to ptr
inst_spare_word       ds 1       * makes names regular
```

An instrument is made up of at least three, and less than 210, objects. The first object is always the inst_header, which contains pointers to the rest. The rest of the objects are either layers or WaveSamples. There may be up to eight layers and up to 127 WaveSamples. All objects keep in the first two words the size of the object and a handle index.

* STRUCTURE INFO

```
inst_name            ds 12
```

* PARAMS

```
inst_midi_chan        ds 1      * use high byte
inst_midi_prog         ds 1      * (4) slave layer's channel
inst_midi_pressure     ds 1      * (5) slave layer's program number
inst_size_blocks       ds 1      * (6)
inst_midi_status       ds 1      * in blocks
inst_midi_status       ds 1      * (8) both,local,midi,ext
inst_programs
inst_program_0         ds 1      * (0)
inst_program_1         ds 1      * (1)
inst_program_2         ds 1      * (2)
inst_program_3         ds 1      * (3)
inst_keydown_layers    ds 1
inst_keyup_layers      ds 1
inst_patch_select      ds 1      * Live,00,0*,*0,**,Held
inst_pedal_down_layers ds 1
inst_id_field          ds 1      * $FFFF for EPS-16 PLUS
                           * otherwise, signifies Original EPS
inst_range_lo          ds 1      * (10) lowest and highest key
inst_range_hi          ds 1      * (11) included in inst range
inst_transpose         ds 1      * (12) hi byte is signed semitones
```

* OFFSETS to all the substructures

```
inst_ptable_ptrs      ds 8*2
inst_layer_ptrs        ds 8*2    * relative to instrument start
inst_ws_ptrs           ds 128*2  *
inst_effect_ptr        ds.l 1    * relative to inst start
```

* SPARE

```
inst_size:            ds.b (16-(*.mod.16)).mod.16    * space for misc
```

* layer definition

aseg
org 0

* STRUCTURE INFO

layer_block_size	ds.l 1	* size fixed at 128 words
layer_ptr_offset	ds 1	* number of layer 0-7 (times 16)
layer_list_tail	ds 1	
layer_list_head	ds 1	* index of first ws in this layer
layer_name	ds 12	*

* PARAMETERS

layer_glide_mode	ds.b 1	* (0)
layer_delay_modamt	ds.b 1	* EPS-16 PLUS ONLY
layer_glide_time	ds.b 1	* (1)
layer_restrike_time	ds.b 1	* EPS-16 PLUS ONLY
layer_legato_layer	ds 1	* (2)
layer_vel_lo	ds 1	* (10)
layer_vel_hi	ds 1	* (3)
layer_ptable	ds 1	* (5)
layer_delay	ds 1	* This is a WORD in EPS-16 PLUS ONLY
	ds.b (16-(*.mod.16)).mod.16	* space for misc
layer_map	ds 88	* 12 bit offsets into ws table in inst header
layer_size:		

* pitch table definition

aseg
org 0

* STRUCTURE INFO

ptable_block_size	ds.l 1	* size fixed at 128 words
ptable_ptr_offset	ds 1	* number of ptable 0-7 (times 16)
ptable_list_tail	ds 1	
ptable_list_head	ds 1	* index of first ws in this ptable
ptable_name	ds 12	*
ptable_keymap	ds 88	* key maps
	ds.b (16-(*.mod.16)).mod.16	* space for misc
ptable_size:		

 * WAVESAMPLE ENVELOPE structure, which is a sub structure of the ws:

```

    aseg
    org 0

wse_type          ds 1          * envelope defaults
wse_L0            ds 1          * (0) initial
wse_L0_v          ds 1          * (10)
wse_T1           ds 1          * (1) attack
wse_L1           ds 1          * (11) peak
wse_L1_v          ds 1          * (12)
wse_T2           ds 1          * (2) decay1
wse_L2           ds 1          * (13)
wse_L2_v          ds 1          * (14)
wse_T3           ds 1          * (3) decay2
wse_L3           ds 1          * (15)
wse_L3_v          ds 1          * (16)
wse_T4           ds 1          * (4) decay3
wse_L4           ds 1          * (17) sustain
wse_L4_v          ds 1          * (18)
wse_T5           ds 1          * (5)
wse_L5
wse_vel_switch    ds 1          * vel switch & vel select in EPS-16 PLUS
wse_L5_r          ds 1          * release breakpoint relative (+-)
wse_T6           ds 1          * (6)
wse_T1_v          ds 1          * (7)
wse_key_scale     ds 1          * (8)
wse_mode          ds.b 1        * (9) 0=norm,1=fullcycle,2=piano
wsep_curve        ds.b 1        * not used
wse_size

```

```

*****
* ws: WaveSample
*****

    aseg
    org 0

* STRUCTURE INFO
ws_block_size      ds.l 1
ws_ptr_offset      ds 1
ws_flink           ds 1
ws_blink           ds 1
ws_name            ds 12
ws_copy            ds 1      * if non_zero, the ws that has waveaddress
ws_copy_layer      ds 1      * ..if non-zero above, layer that has original

* ENVELOPES
ws_env1            ds.b      wse_size
ws_env2            ds.b      wse_size
ws_env3            ds.b      wse_size

* PITCH PARAMETERS
ws_root_key        ds.b 1      * (0) key num
wsp_vol_mod_curve  ds.b 1
ws_pitch_env1      ds 1      * (1)
ws_pitch_lfo       ds 1      * (2)
ws_pitch_random    ds 1      * (3)
ws_bend_range      ds.b 1      * (4)
wsp_mixer_a_curve  ds.b 1      * (5)
ws_pitch_modsrc    ds.b 1      * (5)
wsp_mixer_a_modsrc ds.b 1      * (5)
ws_fine_tune       ds 1      * (10) signed fraction
ws_pitch_modamt    ds 1      * (11)

* FILTER
ws_filter_mode     ds.b 1      * (0)
wsp_mixer_b_modsrc ds.b 1      * (5)
ws_fc1             ds 1      * (1)
ws_fc2             ds 1      * (2)
ws_fc1_kbdamt      ds 1      * (3)
ws_fc2_kbdamt      ds 1      * (4) SF
ws_fc1_env2        ds 1      * (5)
ws_fc2_env2        ds 1      * (6)
ws_fc1_modsrc      ds.b 1      * (7)
wsp_mixer_a_modamt ds.b 1      * (5)
ws_fc2_modsrc      ds.b 1      * (8)
wsp_mixer_b_modamt ds.b 1      * (5)
ws_fc1_modamt      ds 1      * (10)
ws_fc2_modamt      ds 1      * (11)

```


* VOLUME

ws_volume	ds.b 1	* (0)	
wsp_bus_select	ds.b 1	*	gus bus select table
ws_vol_modsrc	ds.b 1	* (1)	
wsp_pan_modsrc	ds.b 1	*	gus pan_modsrc_table
ws_vol_mod_a	ds 1	* (2)	
ws_vol_mod_b	ds 1	* (3)	
ws_vol_mod_c	ds 1	* (4)	
ws_vol_mod_d	ds 1	* (5)	
ws_pan	ds.b 1	* (6)	old m2 pan
wsp_pan	ds.b 1	*	signed_frac pan for gus
ws_vol_modamt	ds.b 1	* (10)	
wsp_pan_modamt	ds.b 1	*	signed_frac for gus

* LFO

ws_lfo_wave	ds.b 1	
wsp_boost	ds.b 1	
ws_lfo_speed	ds.b 1	* (0)
wsp_lfo_rate_modamt	ds.b 1	* (0)
ws_lfo_depth	ds 1	* (1)
ws_lfo_delay	ds.b 1	* (2)
wsp_lfo_modamt	ds.b 1	* (3)
ws_lfo_modsrc	ds.b 1	* (3)
wsp_lfo_rate_modsrc	ds.b 1	* (3)
ws_lfo_mode	ds 1	* (4)
ws_random_freq	ds 1	* (5)

* WAVESAMPLE

ws_loop_flags	ds 1	* (0)	
ws_start	ds 4	* (1)	0..FFFFFF
			* movep.l gets 32 bit field
ws_end	ds 4	* (2)	0..FFFFFF
			* asr.l #9 to get wordcnt
ws_loop_start	ds 4	* (3)	0..FFFFFF.F
			* asr.l #5 to get fraction
ws_loop_end	ds 4	* (4)	0..FFFFFF.F
* loop position		* (5)	
ws_sample_rate	ds 1	* (6)	* microseconds 1 - 127
ws_lokey	ds.b 1	* (7)	
	ds.b 1		
ws_hikey	ds.b 1	* (10)	* keyrange
	ds.b 1		
ws_loop_modsrc	ds.b 1		
wsp_delay_modamt	ds.b 1		
ws_loop_modamt	ds 1		
ws_loop_modamt2	ds 1		
ws_mod_type	ds 1		
wsp_delay	ds 1		* THIS IS A WORD
			* size = \$120; done, filled up, no more space for misc
ws_size:			

* effect definition

aseg
org 0

* STRUCTURE INFO

effect_block_size	ds.l 1	* size is variable
effect_ptr_offset	ds 1	* set to inst_effect_ptr
effect_ptr_more	ds 1	* used when effect is its own block
effect_version_num	ds 1	* set to 2 after 9/7/90 (clear spares)
effect_name	ds 12	
effect_size	ds.l 1	* total size in bytes, incl. ucode

* INTERNAL STRUCTURE OF EFFECT: word offsets from effect block start

effect_ucode	ds 1	* microcode downloadable
effect_table	ds 1	* ESP-RAM table, or zero
effect_init_code	ds 1	* called before esp runs code
effect_update_task	ds 1	* gets installed as a task
effect_var_1	ds 1	* blocks of fx control parameters
effect_var_2	ds 1	
effect_var_3	ds 1	
effect_var_4	ds 1	
effect_param_list_start	ds 1	* page of parameter descriptions
effect_param_list_last	ds 1	
effect_param_list_current	ds 1	
effect_fx1_name	ds.b 13	
effect_fx2_name	ds.b 13	
effect_fx3_name	ds.b 13	
effect_current_var	ds.b 1	* even
effect_output_gpr_L	ds.b 1	
effect_output_gpr_R	ds.b 1	
effect_num_voices	ds.b 1	* 0=20 , 1=13 , 2=7
	ds.b 1	
effect_keydown_routine	ds 1	
	ds.b 10	* spares

* note, this block is not padded out to be 16 byte chunky!

